

深度学习框架简介和对比

(一) 深度学习框架简介

深度学习采用的是一种“端到端”的学习模式，从而在很大程度上减轻了研究人员的负担。但是随着神经网络的发展，模型的复杂度也在不断提升。即使是在一个最简单的卷积神经网络中也会包含卷积层、池化层、激活层、Flatten 层、全连接层等。如果每次搭建一个新的网络之前都需要重新实现这些层的话，势必会占用许多时间，因此各大深度学习框架应运而生。框架存在的意义就是屏蔽底层的细节，使研究者可以专注于模型结构。目前较为流行的深度学习框架有 Caffe 、 TensorFlow 以及 Pytorch 等。

提纲

- Caffe的特点和概述
- TensorFlow的特点和概述
- 数据流图
- PyTorch的特点和概述
- 三者的比较

Caffe简介

- Caffe, 全称Convolutional Architecture for Fast Feature Embedding。是一种常用的深度学习框架，主要应用在视频、图像处理方面的应用上。caffe是一个清晰，可读性高，快速的深度学习框架。
- Caffe是第一个主流的工业级深度学习工具，专精于图像处理。它有很多扩展，但是由于一些遗留的架构问题，不够灵活，且对递归网络和语言建模的支持很差。对于基于层的网络结构，caffe扩展性不好；而用户如果想要增加层，则需要自己实现 (forward, backward and gradient update)

Caffe的特点

- Caffe的基本工作流程是设计建立在神经网络的一个简单假设，所有的计算都是层的形式表示的，网络层所做的事情就是输入数据，然后输出计算结果。
- 以卷积为例，卷积就是输入一幅图像，然后和这一层的参数（**filter**）做卷积，最终输出卷积结果。每层需要两种函数计算，一种是**forward**，从输入计算到输出；另一种是**backward**，从上层给的**gradient**来计算相对于输入层的**gradient**。
- 这两个函数实现之后，我们就可以把许多层连接成一个网络，这个网络输入数据（图像，语音或其他原始数据），然后计算需要的输出（比如识别的标签）。在训练的时候，可以根据已有的标签计算**loss**和**gradient**，然后用**gradient**来更新网络中的参数。

Caffe的特点

- Caffe是一个清晰而高效的深度学习框架，它基于纯粹的C++/CUDA架构，支持命令行、Python和MATLAB接口，可以在CPU和GPU直接无缝切换。它的模型与优化都是通过配置文件来设置，无须代码。
- Caffe设计之初就做到了尽可能的模块化，允许对数据格式、网络层和损失函数进行扩展。Caffe的模型定义是用Protocol Buffer（协议缓冲区）语言写进配置文件的，以任意有向无环图的形式。
- Caffe会根据网络需要正确占用内存，通过一个函数调用实现CPU和GPU之间的切换。Caffe每一个单一的模块都对应一个测试，使得测试的覆盖非常方便，同时提供python和matlab接口，用这两种语法进行调用都是可行的。

Caffe概述

- Caffe是一种对新手非常友好的深度学习框架模型，它的相应优化都是以文本形式而非代码形式给出。Caffe中的网络都是有向无环图的集合，可以直接定义。

```
name: "dummy-net"  
layers {name: "data" ...}  
layers {name: "conv" ...}  
layers {name: "pool" ...}  
layers {name: "loss" ...}
```

Caffe概述

- 数据及其导数以blobs的形式在层间流动，Caffe层的定义由2部分组成：层属性与层参数，如图所示。
- 图中前4行是层属性，定义了层名称、层类型以及层连接结构（输入blob和输出blob）；而后半部分是各种层参数。Blob是用以存储数据的4维数组。

```
name:"conv1"  
type:CONVOLUTION  
bottom:"data"  
top:"conv1"  
convolution_param{  
    num_output:20  
    kernel_size:5  
    stride:1  
    weight_filler{  
        type:"xavier"  
    }  
}
```

Caffe概述

- 在Caffe模型中，网络参数的定义也非常方便，可以随意设置相应参数。感觉上更像是配置服务器参数而不像是代码。

```
# test_iter specifies how many forward passes the test should carry out.
# In the case of MNIST, we have test batch size 100 and 100 test iterations,
# covering the full 10,000 testing images.
test_iter: 100
# Carry out testing every 500 training iterations.
test_interval: 500
# The base learning rate, momentum and the weight decay of the network.
base_lr: 0.01
momentum: 0.9
weight_decay: 0.0005
# The learning rate policy
lr_policy: "inv"
gamma: 0.0001
power: 0.75
# Display every 100 iterations
display: 100
# The maximum number of iterations
max_iter: 10000
# snapshot intermediate results
snapshot: 5000
snapshot_prefix: "lenet"
# solver mode: CPU or GPU
solver mode: GPU
```

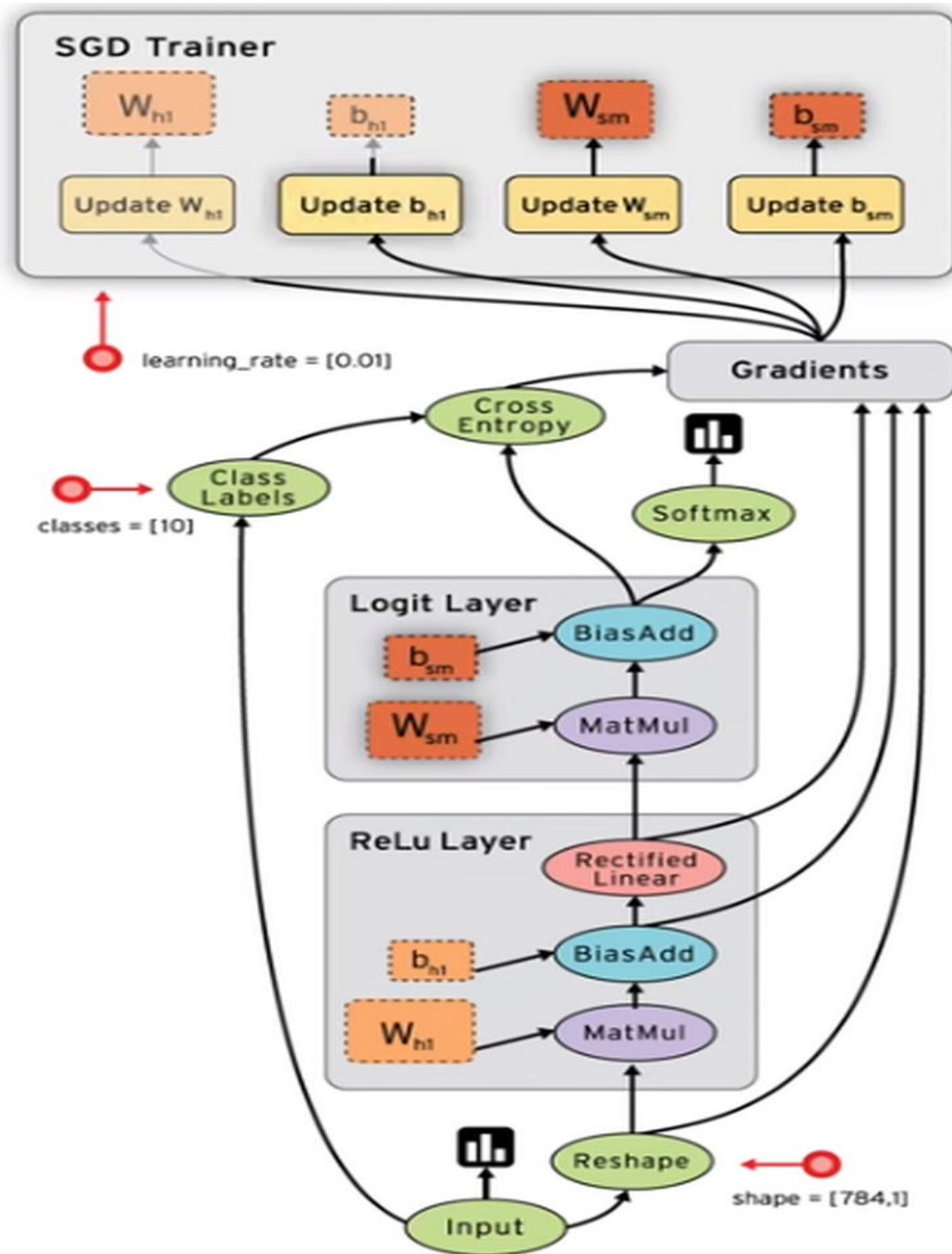
TensorFlow简介

- TensorFlow 是一个采用数据流图 (data flow graphs) 用于数值计算的开源软件库。节点 (Nodes) 在图中表示数学操作，图中的线 (edges) 则表示在节点间相互联系的多维数据数组，即张量 (tensor)。它灵活的架构让你可以在多种平台上展开计算，例如台式计算机中的一个或多个 CPU (或GPU)，服务器，移动设备等等。
- TensorFlow 最初由Google大脑小组的研究员和工程师们开发出来，用于机器学习和深度神经网络方面的研究，但这个系统的通用性使其也可广泛用于其他计算领域。

数据流图

- 数据流图用“结点” (nodes) 和“线”(edges) 的有向图来描述数学计算。“节点”一般用来表示施加的数学操作，但也可以表示数据输入 (feed in) 的起点/输出 (push out) 的终点，或者是读取/写入持久变量 (persistent variable) 的终点。“线”表示“节点”之间的输入/输出关系。这些数据“线”可以输运“size可动态调整”的多维数据数组，即“张量” (tensor)。张量从图中流过的直观图像是这个工具取名为“TensorFlow”的原因。一旦输入端的所有张量准备好，节点将被分配到各种计算设备完成异步并行地执行运算。

数据流图



TensorFlow的特点

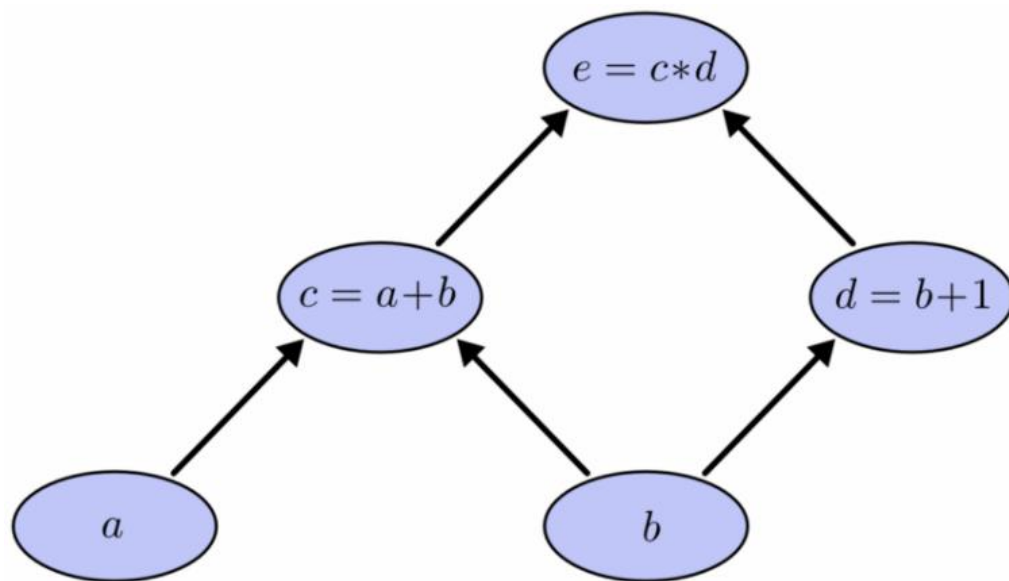
- TensorFlow 不是一个严格的“神经网络”库。只要用户可以将计算表示为一个数据流图就可以使用TensorFlow。用户负责构建图，描写驱动计算的内部循环。TensorFlow提供有用的工具来帮助用户组装“子图”，当然用户也可以自己在TensorFlow基础上写自己的“上层库”。定义新复合操作和写一个Python函数一样容易。TensorFlow的可扩展性相当强，如果用户找不到想要的底层数据操作，也可以自己写一些C++代码来丰富底层的操作。
- TensorFlow 在CPU和GPU上运行，比如说可以运行在台式机、服务器、手机移动设备等等。TensorFlow支持将训练模型自动在多个CPU上规模化运算，以及将模型迁移到移动端后台。

TensorFlow的特点

- 基于梯度的机器学习算法会受益于TensorFlow自动求微分的能力。作为TensorFlow用户，只需要定义预测模型的结构，将这个结构和目标函数（objective function）结合在一起，并添加数据，TensorFlow将自动为用户计算相关的微分导数。计算某个变量相对于其他变量的导数仅仅是通过扩展你的图来完成的，所以用户能一直清楚看到究竟在发生什么。
- TensorFlow 还有一个合理的c++使用界面，也有一个易用的python使用界面来构建和执行你的graphs。你可以直接写python/c++程序，也可以用交互式的ipython界面来用TensorFlow尝试些想法，它可以帮用户将笔记、代码、可视化等有条理地归置好。

TensorFlow概述

- TensorFlow中Flow，也就是流，是其完成运算的基本方式。流是指一个计算图或简单的一个图，图不能形成环路，图中的每个节点代表一个操作，如加法、减法等。每个操作都会导致新的张量形成。



TensorFlow概述

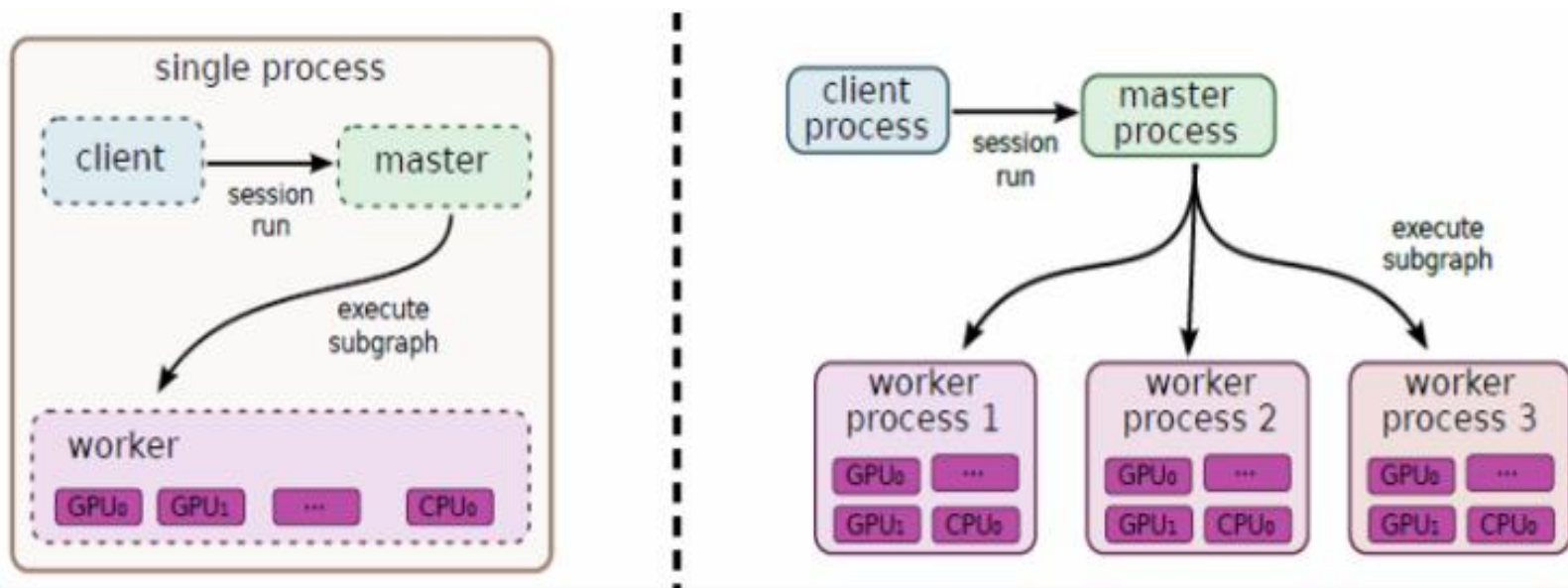
- 计算图具有以下属性：叶子顶点或起始顶点始终是张量。意即，操作永远不会发生在图的开头，由此可以推断，图中的每个操作都应该接受一个张量并产生一个新的张量。同样，张量不能作为非叶子节点出现，这意味着它们应始终作为输入提供给操作/节点。
- 计算图总是以层次顺序表达复杂的操作。通过将 $a + b$ 表示为 c ，将 $b + 1$ 表示为 d ，可以分层次组织上述表达式。因此，可以将 e 写为： $e = (c) \times (d)$ 这里 $c = a + b$ 且 $d = b + 1$. 以反序遍历图形而形成子表达式，这些子表达式组合起来形成最终表达式。

TensorFlow概述

- 当我们正向遍历时，遇到的顶点总是成为下一个顶点的依赖关系，例如没有a和b就无法获得c，同样的，如果不解决c和d则无法获得e。同级节点的操作彼此独立，这是计算图的重要属性之一。
- 当我们按照图中所示的方式构造一个图时，很自然的是，在同一级中的节点，例如c和d，彼此独立，这意味着没有必要在计算d之前计算c。因此它们可以并行执行。
- 计算图的并行也是最重要的属性之一。它清楚地表明，同级的节点是独立的，这意味着在c被计算之前不需空闲，可以在计算c的同时并行计算d。
TensorFlow充分利用了这个属性。

TensorFlow概述

- TensorFlow允许用户使用并行计算设备更快地执行操作。计算的节点或操作自动调度进行并行计算。这一切都发生在内部，例如在上图中，可以在CPU上调度操作c，在GPU上调度操作d。下图展示了两两种分布式执行的过程。

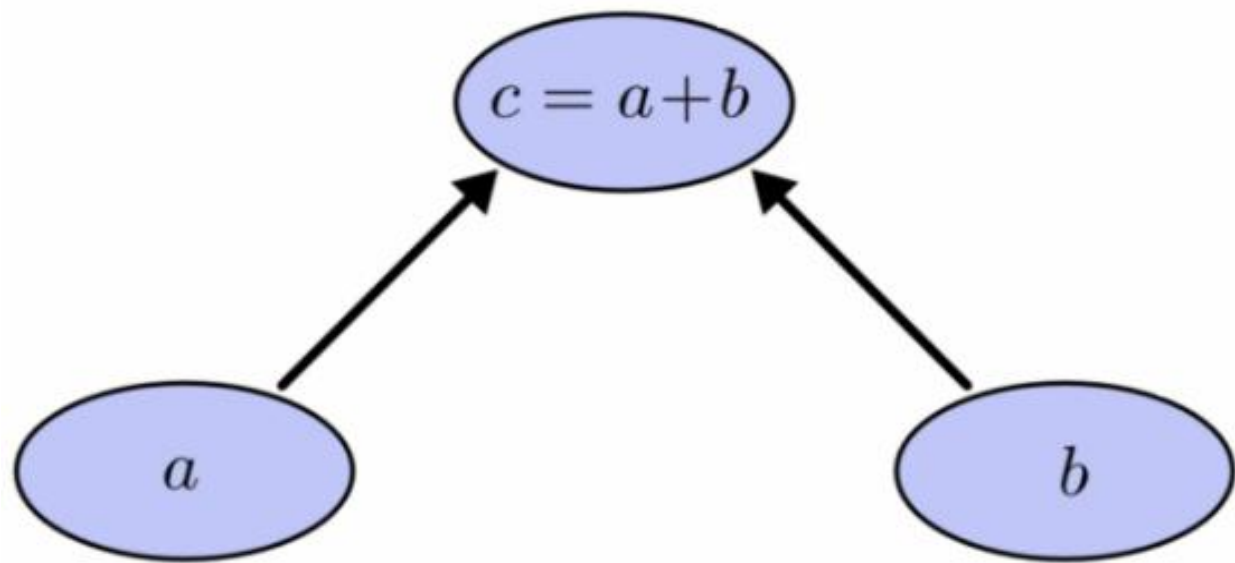


TensorFlow概述

- 上图中，第一种是单个系统分布式执行，其中单个TensorFlow会话创建单个worker，并且该worker负责在各设备上调度任务。在第二种系统下，有多个worker，他们可以在同一台机器上或不同的机器上，每个worker都在自己的上下文中运行。在上图中，worker进程1运行在独立的机器上，并调度所有可用设备进行计算。

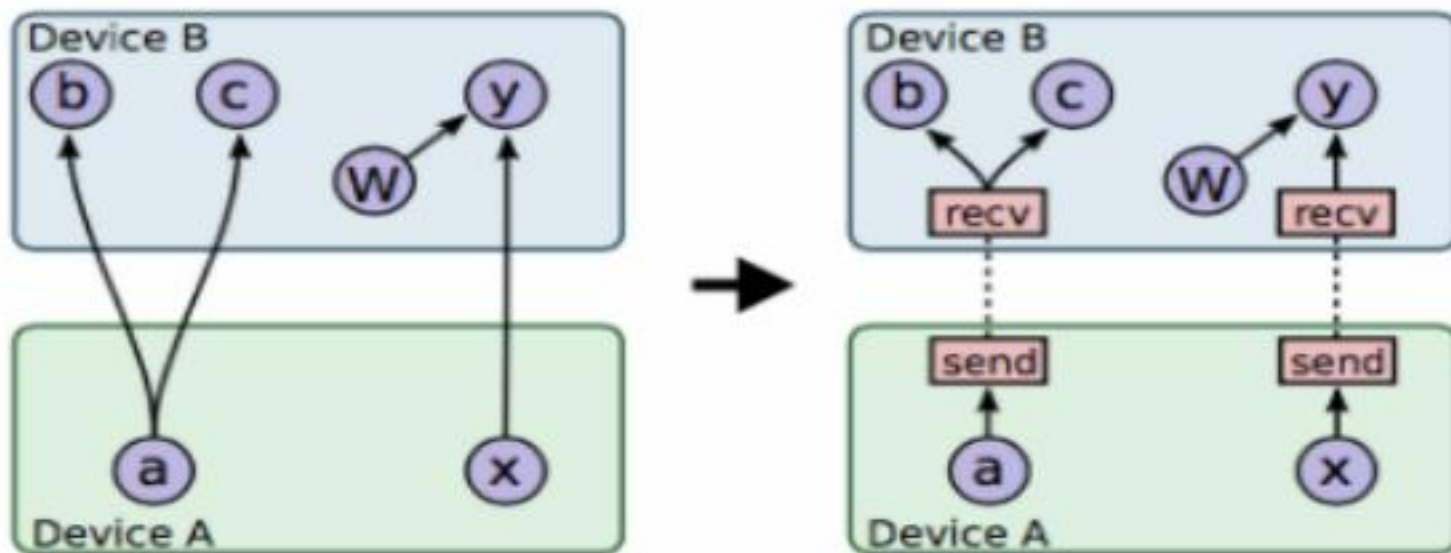
TensorFlow概述

- 子图是主图的一部分，从属性2可以说子图总是表示一个子表达式，因为 c 是 e 的子表达式。子图也满足最后一个属性。同一级别的子图也相互独立，可以并行执行。因此可以在一台设备上调度整个子图。



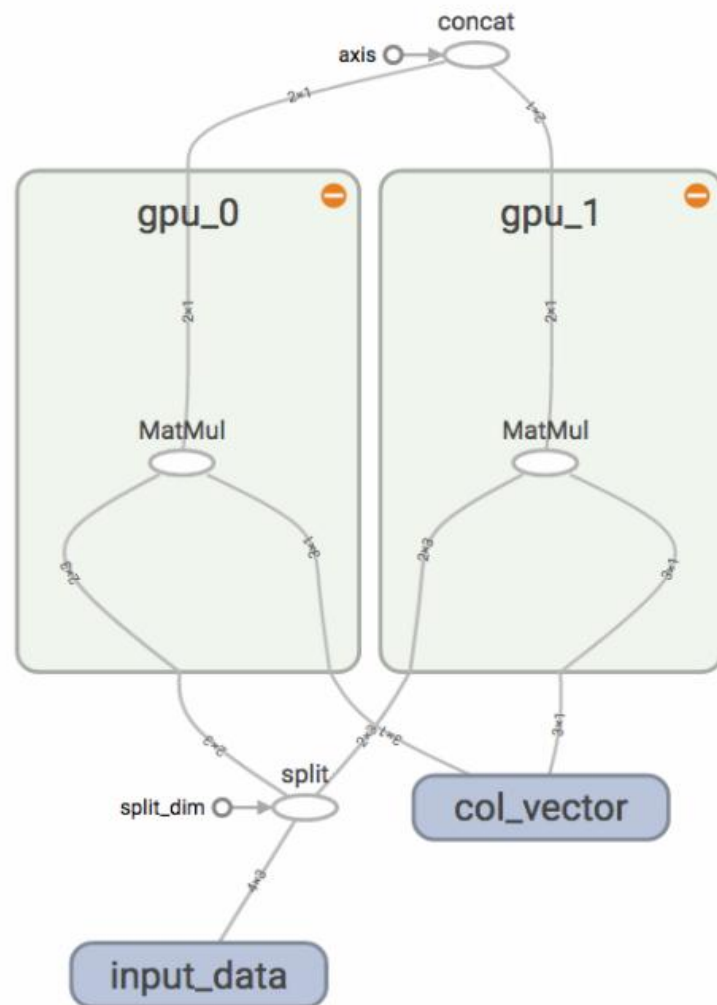
TensorFlow概述

- TensorFlow将其所有操作分配到由worker管理的不同设备上。更常见的是，worker之间交换张量形式的数据，例如在 $e = (c) * (d)$ 的图表中，一旦计算出 c ，就需要将其进一步传递给 e ，因此Tensor在节点间前向流动。该流动如下图所示



TensorFlow概述

- 右图解释了子图的并行执行。这里有2个矩阵乘法运算，因为它们都处于同一级别，彼此独立，这符合最后一个属性。由于独立性的缘故，节点安排在不同的设备gpu_0和gpu_1上。



PyTorch简介

- PyTorch具有先进设计理念的框架，其历史可追溯到2002年就诞生于纽约大学的Torch。Torch使用了一种不是很大众的语言Lua作为接口。Lua简洁高效，但由于其过于小众，以至于很多人听说要掌握Torch必须新学一门语言就望而却步，但Lua其实是一门比Python还简单的语言。
- 考虑到Python在计算科学领域的领先地位，以及其生态完整性和接口易用性，几乎任何框架都不可避免地要提供Python接口。终于，在2017年，Torch的幕后团队推出了PyTorch。PyTorch不是简单地封装Lua Torch提供Python接口，而是对Tensor之上的所有模块进行了重构，并新增了最先进的自动求导系统，成为当下最流行的动态图框架。

PyTorch的特点

- PyTorch 既可以看作为加入了GPU 支持的numpy。TensorFlow 与Caffe 都是命令式的编程语言，而且是静态的，即首先必须构建一个神经网络，然后一次又一次使用同样的结构；如果想要改变网络的结构，就必须从头开始。但是PyTorch通过一种反向自动求导的技术，可以让用户零延迟地任意改变神经网络的行为。
- PyTorch 的设计思路是线性、直观且易于使用的，当用户执行一行代码时，它会忠实地执行，所以当用户的代码出现Bug 的时候，可以通过这些信息轻松快捷地找到出错的代码，不会让用户在Debug 的时候因为错误的指向或者异步和不透明的引擎浪费太多的时间。
- PyTorch 的代码相对于TensorFlow 而言，更加简洁直观，同时对于TensorFlow高度工业化的底层代码，PyTorch 的源代码就要友好得多，更容易看懂。深入API，理解PyTorch 底层肯定是一件令人高兴的事。

PyTorch概述

- PyTorch 最大优势是建立的神经网络是动态的，可以非常容易的输出每一步的调试结果，相比于其他框架来说，调试起来十分方便。
- PyTorch的图是随着代码的运行逐步建立起来的，也就是说使用者并不需要在一开始就定义好全部的网络的结构，而是可以随着编码的进行来一点一点的调试，相比于TensorFlow和Caffe的静态图而言，这种设计显得更加贴近一般人的编码习惯。

PyTorch概述

A graph is created on the fly

```
from torch.autograd import Variable
```

```
x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))
```

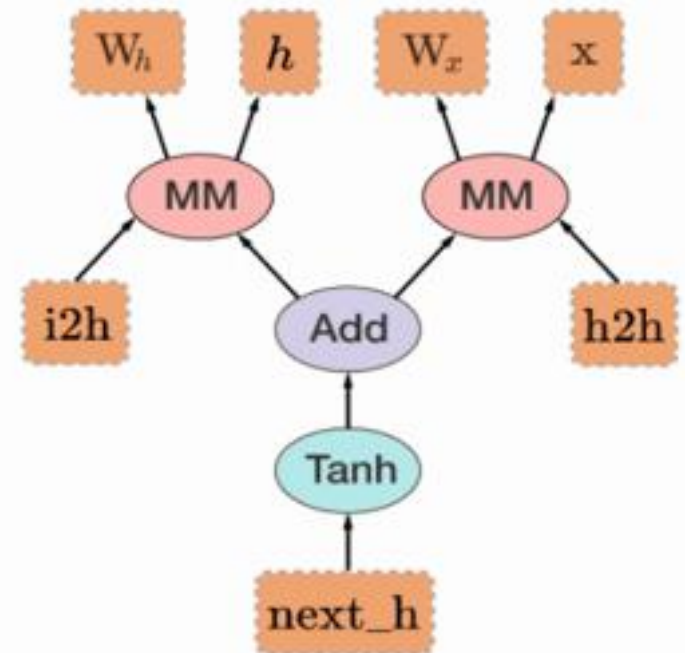
Back-propagation
uses the dynamically built graph

```
from torch.autograd import Variable
```

```
x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))
```

```
i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()
```

```
next_h.backward(torch.ones(1, 20))
```



三者的比较

- Caffe

- Caffe的优点是简洁快速，缺点是缺少灵活性。Caffe灵活性的缺失主要是因为它的设计缺陷。
- Caffe凭借其易用性、简洁明了的源码、出众的性能和快速的原型设计获取了众多用户。但是在深度学习新时代到来之时，Caffe已经表现出明显的力不从心，诸多问题逐渐显现，包括灵活性缺失、扩展难、依赖众多环境难以配置、应用局限等。尽管现在在GitHub上还能找到许多基于Caffe的项目，但是新的项目已经越来越少。

三者的比较

- Caffe2

- Caffe的作者加入FAIR后开发了Caffe2。Caffe2是一个兼具表现力、速度和模块性的开源深度学习框架。它沿袭了大量的 Caffe 设计，可解决多年来在 Caffe 的使用和部署中发现的瓶颈问题。
- Caffe2的设计追求轻量级，在保有扩展性和高性能的同时，Caffe2 也强调了便携性。Caffe2 从一开始就以性能、扩展、移动端部署作为主要设计目标。
- Caffe2继承了Caffe的优点，在速度上令人印象深刻。然而尽管已经发布半年多，开发一年多，Caffe2仍然是一个不太成熟的框架。

三者的比较

- TensorFlow

- TensorFlow已经成为当今最炙手可热的深度学习框架，但是由于自身的缺陷，TensorFlow离最初的设计目标还很遥远。
- TensorFlow和Theano拥有相近的设计理念，都是基于计算图实现自动微分系统。TensorFlow 使用数据流图进行数值计算，图中的节点代表数学运算，而图中的边则代表在这些节点之间传递的多维数组。
- TensorFlow编程接口支持Python和C++。随着1.0版本的公布，Java、Go、R和Haskell API的alpha版本也被支持。此外，TensorFlow还可在Google Cloud和AWS中运行。TensorFlow还支持 Windows 7、Windows 10和Windows Server 2016。由于TensorFlow使用C++ Eigen库，所以库可在ARM架构上编译和优化。这也就意味着用户可以在各种服务器和移动设备上部署自己的训练模型，无须执行单独的模型解码器或者加载Python解释器。

三者的比较

- TensorFlow的批评

- 过于复杂的系统设计。TensorFlow 代码量庞大，使维护难以完成。对于入门者，TensorFlow的底层运行机制难以掌握，很多时候导致了放弃。
- 频繁变动的接口。TensorFlow的接口一直处于快速迭代之中，且向后兼容性较差，这导致许多陈旧的开源代码已经无法在新版运行，同时也间接导致了許多第三方框架出现BUG。
- 接口设计过于晦涩难懂，拥有图、会话、命名空间、Placeholder等诸多抽象概念，对普通用户来说难以理解。同一个功能，TensorFlow提供了多种实现，这些实现良莠不齐，使用中还有细微的区别，很容易将用户带入坑中。
- TensorFlow作为一个复杂的系统，文档和教程众多，但缺乏明显的条理和层次，虽然查找很方便，但用户却很难找到一个真正循序渐进的入门教程。
- 至今TensorFlow仍没有一个统一易用的接口。

三者的比较

- PyTorch

- PyTorch是当前难得的简洁优雅且高效快速的框架。PyTorch的设计追求最少的封装，遵循 `tensor`→`variable`(`autograd`)→`nn.Module` 三个由低到高的抽象层次，分别代表高维数组（张量）、自动求导（变量）和神经网络（层/模块），而且这三个抽象之间联系紧密，可以同时进行修改和操作。
- 简洁的设计带来的另外一个好处就是代码易于理解。
- PyTorch的灵活性不以速度为代价，在许多评测中，PyTorch的速度表现胜过TensorFlow和Keras等框架。

三者的比较

- PyTorch

- PyTorch正在受到越来越多人的追捧。
- PyTorch是所有的框架中面向对象设计的最优雅的一个。PyTorch的面向对象的接口设计来源于Torch，而Torch的接口设计以灵活易用而著称。PyTorch继承了Torch的衣钵，尤其是API的设计和模块的接口都与Torch高度一致。PyTorch的设计最符合人们的思维，它让用户尽可能地专注于实现自己的想法，即所思即所得，不需要考虑太多关于框架本身的束缚。
- PyTorch提供了完整的文档，循序渐进的指南，作者亲自维护的论坛供用户交流和请教问题。Facebook人工智能研究院对PyTorch提供了强力支持，作为当今排名前三的深度学习研究机构，FAIR的支持足以确保PyTorch获得持续的开发更新。
- 同样的，由于推出时间较短，在Github上并没有如Caffe或TensorFlow那样多的代码实现，使用TensorFlow能找到很多别人的代码，而对于PyTorch的使用者，可能需要自己完成很多的代码实现。

小结

- 本章介绍了三种常用的机器学习框架，其中TensorFlow和PyTorch是目前最流行的两种开源框架。在以往版本的实现中，TensorFlow主要提供静态图构建的功能，因此具有较高的运算性能，但是模型的调试分析成本较高。PyTorch主要提供动态图计算的功能，API涉及接近Python原生语法，因此易用性较好，但是在图优化方面不如TensorFlow。这样的特点导致TensorFlow大量用于AI企业的模型部署，而学术界大量使用PyTorch进行研究。不过目前我们也看到两种框架正在吸收对方的优势，例如TensorFlow的eager模式就是对动态图的一种尝试。